Socialising
COMP1161 Project

Dr. Phillipa Bennett

Contents

List of Figures

Listings

1 Introduction to Socialise (and Social Networking)

A Google Oxford Language dictionary defines a social network (SN) as:

1. a network of social interactions and personal relationships.
2. a dedicated website or other application which enables users to communicate with each other by posting information, comments, messages, images, etc.

A brief history of SNs may be found at

`https://www.britannica.com/technology/social-network`.

Even Wikipedia has some very useful information about SNs, see

`https://en.wikipedia.org/wiki/Social_network`.

and for SN services, see

`https://en.wikipedia.org/wiki/Social_networking_service`

Socialise is a minimal and generic social network. Socialise represents a static view of a social network, i.e., its objects and collection of objects in other objects represent a view of the system at a particular point in time.

2 Requirement for Socialise

Socialise has the following requirements:

1. when users join, they must give a unique username;
2. when users add posts, Socialise also keeps a list of the posts – this will help when it is time to determine which posts should be on a user's page;
3. posts are shared publicly in the Socialise, or with followers of a user;
4. posts are of two types: a regular post or a post that is a reshare — the differences are:
   a) a regular post may have contents that is an ordering of a combination of text and an image file; and
   b) a reshare post does not have contents, it links to previous post – keep in mind that the linked post should be accessible to the user who reshared it and a post cannot reshare itself;
5. a reshare post can only be accessible to the users who could see the original post — for example,
   a) if `user1` creates post `p1` and shares it with their followers, and `user2` creates a post `p2` that reshares `p1` and shares it also with their followers, only the users in the intersection of `user1`'s and `user2`'s followers should be able to access `p2`;
   b) if `user1` creates post `p1` and shares it with all the members of the Socialise, and `user2` creates a post `p2` that reshares `p1` and shares it with their followers, only the users in `user2`'s followers should be able to access `p2`; and
   c) if `user1` creates post `p1` and shares it with their followers, and `user2` creates a post `p2` that reshares `p1` and shares it also with all the members in the Socialise, only the users in the intersection of `user1`'s and `user2`'s followers should be able to access `p2`.
6. a user may follow another user;
7. a user may add a reaction to a post that is accessible to them; the reactions are of two types *upvotes* or *downvotes*;
8. a user may not react to their own post;
9. a user may react on a post only once; and
10. for a user's page, the posts should appear sorted according to the sort algorithm of the Socialise; a sort algorithm of:
    - *Popular* sorts the posts according to their popularity score: the difference of the sum of upvotes and downvotes;
    - *Oldest* sorts the posts according to the order in which they were posted; and

- *Newest* sorts the posts according to the reverse order in which they were posted.

To help with self governance, always keep in mind that a class must maintain control over its instance data, therefore, for this project, getters in a class should not return the pointer that is a collection to any of their instance data.

Also, for this project, once a post is created or contents added to it, they are not mutable; i.e., no change allowed, however the post may be deleted.

3 Classes

3.1 Initial Design

The initial design for Socialise is shown in the UML class diagram in Figure 1.

While not represented in the diagram, all the inherited attributes should have protected visibility, all the other attributes should have private visibility, and all the methods should have public visibility.

A starting set of Java files is also provided with some methods already implemented — in it you will see some other accessors, mutators, and helper methods and classes in each class. Unless otherwise indicated/advised/instructed:

1. do not change the method headers including visibility modifiers or the return types on these methods;
2. if the method has already been fully/correctly implemented, do not change the implementations of these methods;
3. the `User` class should implement the `Comparable` interface to compare users based on username based;
4. if a method should search a collection and return a value from the collection, if the value is not found, return a null reference for return object types and -1 for return integer types;
5. methods that return ordered collections must return the items sorted according to the context of the method call; and
6. no other attributes (nor should their declared types be changed) or public methods should be added to the classes.

3.2 Packages

The package structure must adhere to the following.

1. all the enumerations are to be placed in a package called `project/enums` — this means that the first line of each file must have the following line:

   ```
   package project.enums;
   ```

2. the `Displayable` interface should be placed in a package called `project/interfaces` — This means that the first line of each file must have the following line:

   ```
   package project.interfaces;
   ```

   and

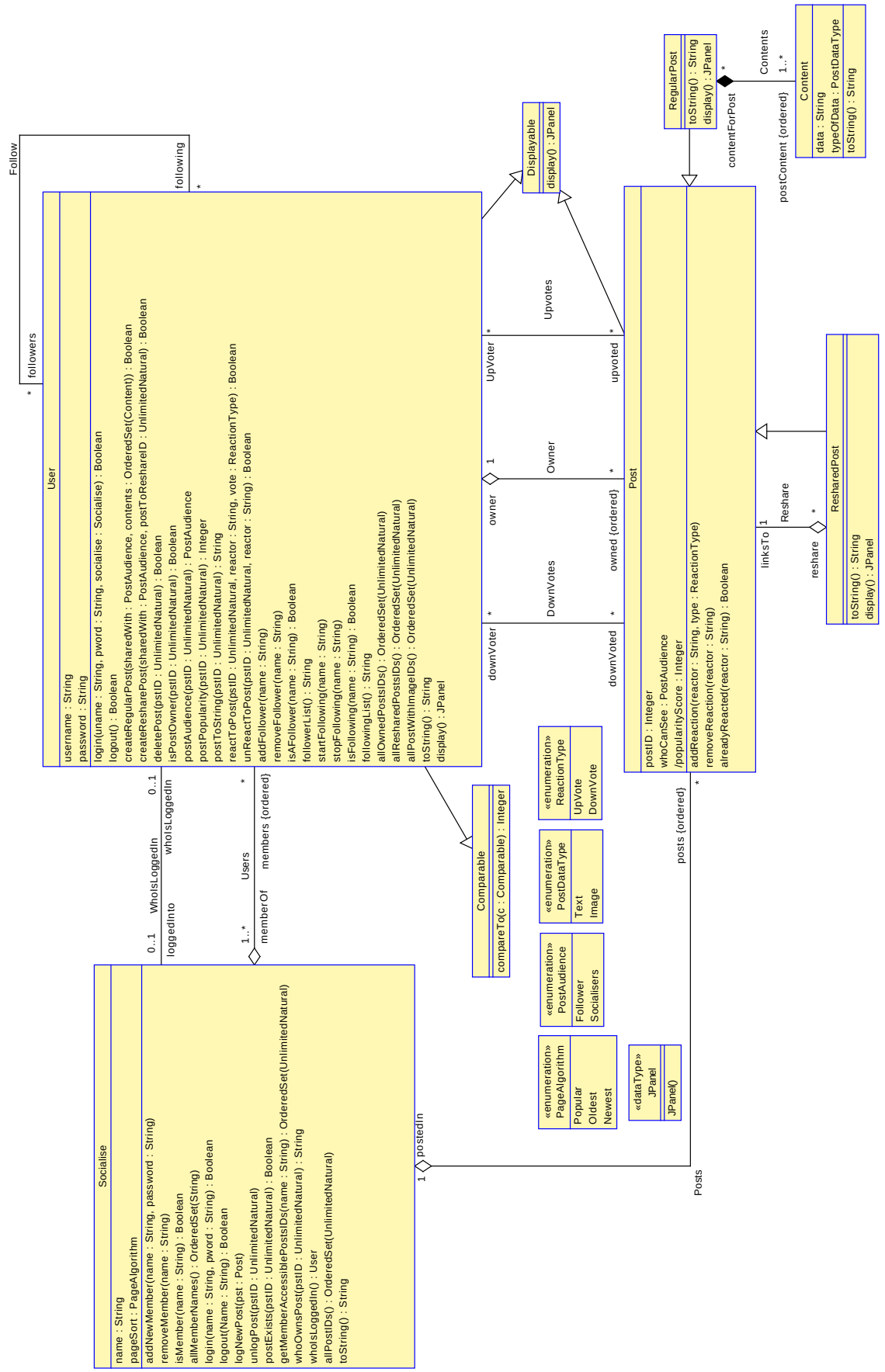

Figure 1: Class diagram showing the design for Socialise

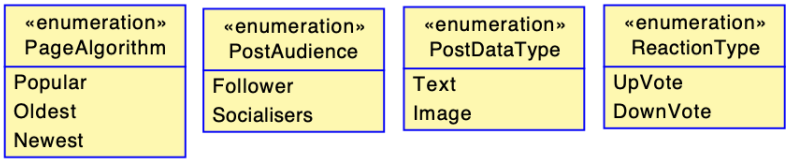

Figure 2: Enumerations

3. all the other classes are to be placed in a package called `project` — this means that the first line of each file must have the following line:

```
package project;
```

3.3 Standard UML and Java Equivalences

Figure 1 uses standard UML Class Model declarations and types; for this project their equivalences in the Java language are as follows, the UML:

1. *Integer* type becomes any of the integer primitive types in Java and one should choose the type based on the range of values that is required in your domain;
2. *UnlimitedNatural* type becomes any of the integer primitive types in Java with the added constraint of being non-negative;
3. *Real* type becomes the any of the real valued primitive types in Java and you choose the type based on the range of values and the precision required in your domain;
4. *Boolean* type becomes the `boolean` primitive type in Java; and
5. *Set*, *OrderedSet*, or *Sequence* types on a collection or *ordered* on a role denotes a collection or ordering or sequencing of the values. For these in this project, we will use the `ArrayList` type; for example, the UML *OrderedSet(String)* would be the `ArrayList<String>` and *Sequence(String)* would be the `ArrayList<String>` in Java. The former, by its definition, should have no duplicates and the latter is allowed to hold duplicates.

In addition to the attributes in the classes, UML diagrams (should) show attributes that are collections as roles on the association lines. These collections will be implemented as `ArrayList` of the associated type in Java. Not all the roles will be used in this project — the ones used are included in the starting specifications.

You should keep in mind that ArrayLists allow duplicates but the collections in this project should not have duplicates, whether by aliases or by using non-unique identifiers.

3.4 Enumerations

The enumerations are (again) shown in Figure 2. Each literal in each enumeration type will have a description attribute.

3.4.1 Example for `PostAudience`

As an example, the definition for the `PostAudience` enumeration is given in Listing 1 where both a constructor and an accessor (getter) method are included.

Listing 1: Java Code for the PostAudience enumeration

```
package project.enums;

public enum PostAudience {

  Followers("Only Followers"), Socialisers("Anyone in the current Socialiser");

  private String description;

  PostAudience(String description) {
    this.description = description;
  }

  public String getDescription() {
    return description;
  }

}
```

All the other enumerations are similarly defined.

3.5 Imports

The following imports of class libraries may be useful to you:

1. `import java.util.ArrayList;`

   This class library is mainly used to store collections. See
   `https://docs.oracle.com/javase/8/docs/api/?java/util/ArrayList.html`
   for its JavaDoc documentation.

2. `import java.util.Arrays;`
   This class library is mainly used to create or sort collections. Keep in mind your `int compareTo(...)` method in the associated classes must be implemented correctly for the sort to produce correct results.

   As an example, suppose you have the following variable defined as:

   ```
   ArrayList<String> stringList;
   ```

and you want to initialise it with a list of strings, then you may use:

```
new ArrayList<String>(Arrays.asList(new String[] {"Cross Roads", "Half Way Tree", "Downtown"})
```

In another example, suppose you have the following:

```
ArrayList<String> names;
```

where the `String` class already implements the `Comparable<String>` interface, then to sort the `names`, you may use the following statements:

```
if (names.size()> 1){
  Object[] z = names();
  Arrays.sort(z);
  names = new ArrayList(Arrays.asList(z));
}
```

Alternatively, you could use the other form of the `toArray()` method as follows:

```
if (names.size()> 1){
   String[] acc1 = names(new String[0]);
   Arrays.sort(acc1);
   List<Station> ss = Arrays.asList(acc1);
}
```

and use `ss` in your computations.

See
`https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html`
for the JavaDoc documentation for the `Arrays` class library.

3. `import java.util.Iterator;`

   You may use the class library in much the same way as you would iterators like `Scanner` objects.

   As an example, suppose you have the following variable defined as:

   ```
   ArrayList<String> stringList;
   ```

   you may use:

```
Iterator<String> sgs = stringList();
while (sgs.hasNext()){
  // code that includes the use of sgs
}
```

to process the elements of the collection.

See

`https://docs.oracle.com/javase/8/docs/api/?java/util/Iterator.html`

for its JavaDoc documentation.

4. `import java.util.Comparator;` and `import java.util.Collections;` to use the `Comparator` interface. This project requires the sorting of `Posts` in different ways. To do this, the `Comparator` interface may be used. See

   `https://docs.oracle.com/javase/8/docs/api/?java/util/Comparator.html`

   for its JavaDoc documentation.

   A section of the starting code for the `Socialise` class is shown in Listing 2. Included in it are two internal classes, `SortByID` and `SorByPopularity`.

   Listing 2: A section of the starting Java code for the Socialise class

```
public class Socialise {
  private ArrayList<Post> posts;

  // sections removed

  class SortByID implements Comparator<Post> {
    public int compare(Post a, Post b) {
      return ((Integer) a.getPostID()).compareTo(b.getPostID());
    }
  }

  class SortByPopularity implements Comparator<Post> {
    public int compare(Post a, Post b) {
      return ((Integer) a.getPopularityScore()).compareTo(b.getPopularityScore());

    }
  }
}
```

   To use any of these comparators, pass the specific comparator as an argument into a sorting method from the Collections class. For example,

```
Comparator myComparator = new SortByPopularity();
Collections.sort(posts, myComparator);
```

   will sort the posts using the popularity score in ascending order.

4 Tasks for Project

1. *This task is worth 30% of the marks.* Using the document at
   `https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html`
   as a guide, provide JavaDoc comments for all of the methods (including the methods in the enumerated types). At a minimum, you must add comments to:

   a) describe the method;

   b) describe each of the parameters (use the `@params` keyword for each parameter); and

   c) if the method has a return type, describe how to interpret or use the return type (use the `@returns` or `@return`).

   Keep in mind that:

   a) JavaDoc comments are enclosed using a double asterisk at the beginning of the comment, like this

   ```
   /** This is a JavaDoc comment */
   ```

   instead of a single asterisk in

   ```
   /* This is a regular comment */
   ```

   for other kinds of comments that are put in a Java file.

   b) writing the JavaDoc will help you to go through and understand the starting code.

   c) when you are reviewing the code, keep in mind that given the following declaration:

   ```
   ArrayList<Station> names
   ```

   the statement

   ```
   names.forEach(s-> System.out.println(s))
   ```

   Could be used as an equivalent form of:

   ```
   for (String s: names)
      System.out.println(s)
   ```

2. *This task is worth 40% of the marks.*

   Implement all the methods in all the classes in the starting specifications.

   Where guidance has not been provided, the project is testing your ability to reason about what should be there based on the information you have been provided in the starting code and the descriptions in this document.

   The method names are fairly descriptive and should give you insight into what is required. If you are not clear on what a method should do, please ask the teacher.

   Please ensure that you use the method names as given, the parameter names, types and order as given, and the return types as given in the class diagram (specifically their corresponding types in Java as described previously in this document).

   We will use automated testing to judge the correct implementation of the methods.

3. *This task is worth 30% of the marks*

   a) Implement a user interface (GUI) for the application – if you implement a partial GUI you could lose up 90% of the marks assigned to this task. If you do not implement a GUI you will lose all the marks assigned to this task.

      Add your GUI classes to the `project.gui` package.

   b) The GUI classes in your application must be passed the same instance of the Socialise in their constructors.

   c) Your GUI should allow the calling of any of the public methods available in the `Socialise` or the `User` classes, for the latter, when a user logs in the GUI should be passed a pointer to the user..

   d) You will be marked based on whether you clearly define what is required from the user, checking for invalid input by telling the user about it, etc, and catching exceptions as needed.

4. Demonstrate your Project.

   a) The signup for slots will be available in week 12, the demonstrations will be done in week 13, and the demos can be facilitated online.

   b) The marks for tasks 1 and 3 will be assigned in the demonstration – you will demonstrate the generation of your javadoc documentation and answer questions about what you did, and you will demonstrate your GUI.

   c) If you do not demonstrate your project you will have forfeited the marks that will be assigned at the demonstration.

   d) All the group's members must be present at the demonstration of your submission.

5 Addendum to Project

Corrections and clarifications after the initial publishing of this document will be added here.

5.1 November 11, 2024

Initial document available.